

PATH Supplement - Status and Progress of Heterogeneous Model Training

October 13, 2025

1 Executive Summary

This project investigates the feasibility and experience of training machine learning model ensembles across heterogeneous resources from a single HTCondor access point (AP). These resources are spatially separated and exist across multiple computing ecosystems, using both the OSPool and resources granted through the NAIRR Pilot Program. We trained 21 variants of a METL protein language model, consuming over 5,300 GPU hours on 6 different GPU models. Our results show that model training is robust in a heterogeneous environment, and that the PATH services, backed by the HTCondor and Pelican software, provide suitable tools for submitting and managing the training process. However, there are workflow challenges that have been identified as well, especially in terms of full automation. Performance variability, especially across similar GPU models, indicate site-specific factors that can negatively impact training performance. The AWS NAIRR resources provided via Cloudbank continue to be unavailable, as security restrictions and default configurations provided by AWS restrict the ability to create the necessary resources for integration into the PATH ecosystem. Overall, this work shows that the PATH ecosystem is technically viable solution for training ensembles of models requiring that are easily checkpointable and fit within a single GPU.

2 Objectives

As a supplement to the Partnership to Advanced Throughput Computing (PATH) project, we have launched a collaboration with a group of domain scientists and ML researchers to run a pathfinder project profiling the effects of throughput training and inference on distributed, heterogeneous capacity. The goals of this activity, are threefold:

1. Characterize the impact of training ensembles across heterogeneous resources, as opposed to the traditional approach of “single homogeneous cluster”.
2. Use the workloads developed in (1) to improve capabilities and services, reducing the barrier of entry for new ML researchers to use the PATH services to train ensembles with capacity powered by the NAIRR pilot.
3. Demonstrate running single workloads managed by a single Access Point (AP) effectively across as many of the NAIRR pilot resources as possible.

To accomplish these goals, we will use the existing Gitter lab’s published Mutational Effect Transfer Learning (METL) protein language model[1], using existing data and software to train a modest number of model variants (20-30) from a single AP, intentionally moving the training processes between available resources at training epoch boundaries. Comparison of these variants will prove compatibility of the models, ensuring that the heterogeneity of resources did not impact the scientific power of the models.

3 Methodology

3.1 NAIRR allocations

Through the NAIRR pilot program, we applied (NAIRR240335) for and received allocations at 6 sites:

Site	Approved allocation
NCSA Delta GPU	11,500 GPU hours
Amazon Web Services	\$6,300
Indiana Jetstream2 GPU	86,016 SUs
PSC Bridges-2 GPU	1,500 GPU hours
Purdue Anvil GPU	1,500 GPU hours
SDSC Expanse GPU	1,500 GPU hours

3.2 Access Point

We used the OSPool access point `ap40.uw.osg-htc.org` for this initial round of experiments. In parallel, a machine dedicated for the PATH supplement was ordered and is currently being configured as of 2025-10-13). There is no special configuration needed on this machine to enable utilization of the NAIRR resources beyond the pre-existing configuration to enable HTCondor Annex creation.

3.3 Job description

The METL software is well-documented with publicly available sourcecode (<https://github.com/gitter-lab/metl/>). Input data and conda environments are similarly available within Zenodo[2]. In order to ease the distribution and deployment of the training software, a Docker container image was created (<https://github.com/CHTC/METL-Conductor/blob/main/Dockerfile>) which contains the installed software. This image was then converted into an apptainer[3] image. The apptainer and input data were then stored within the OSDF to ease the movement across the computing ecosystem. Total input sizes are ~6GB for the image and ~20GB for the data.

Initial benchmarking of the training task was first tested locally on V100 machines, giving approximate resource needs. Additional benchmarking was done both in the CHTC and OSPool clusters to further test and understand the performance and resource needs of the training process.

Finally, test jobs were run within the targeted NAIRR sites to comb out the differences between the sites and to dial in the resource requirements within the job definitions needed for each. An initial training model (uuid 861e7e66) was trained, alternating between Expanse and Delta annexes.

3.4 HTCondor Annex

In order to provide single-AP access to the NAIRR sites, we used HTCondor’s Annex feature. This tool submits a pilot job (or jobs) into a remote site based on the resource requests (and desired number of nodes) from the researcher. These jobs leverage the sites’ local scheduling software and, upon starting, will create an HTCondor Execution Point (EP) that become available within the researcher’s local cluster to run their job lists.

Minor development was needed within the annex software, primarily to update the partition definitions and settings at the remote sites. The AWS annex required additional development to run an Amazon Machine Image (AMI) compatible with GPU usage. As of 2025-10-13, we are still working with Cloudbank to ensure that the AWS user account created for NAIRR allocations have sufficient permissions to create and use the necessary resources within AWS.

3.5 DAG

Building upon a previous summer fellow’s work, we create a DAG generator script, in order to:

1. Organize and automate the workload
2. Identify common patterns of scripted DAG generations
3. From 2, understand the shortcomings of DAGMan within the context of throughput training tasks (ensemble training, hyperparameter sweeps)

The DAG itself is an array of “shish-kebab” subgraphs, with each node defining a single job, itself defined as a single epoch of training. Evaluation of validation data was done within the training job, though the generation script supports separate side-nodes for evaluation.

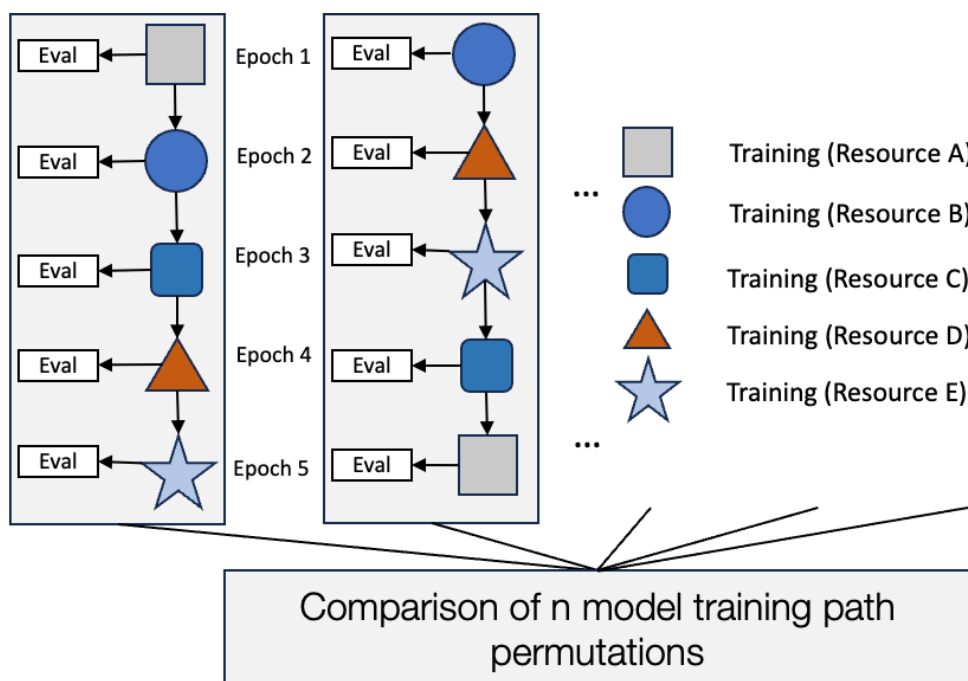


Figure 1: A diagram showing the shape of the DAG implemented. The diagram shows 2 training runs, each of 5 epochs, for simplicity.

3.5.1 Monitoring

In order to monitor the training processes, a script was developed that reads HTCondor logs, DAGMan logs, and the DAG definition files to provide an overview of how training runs are progressing. By default, it shows the most recent (in-progress or idle) epoch within each training run, displaying the HTCondor job ID, the targeted resource, its current state (idle, transferring, running, held, complete), and the time spent in the current state. It also provides a global summarization of the total number of epochs trained across all runs, along with the number of epochs in each state. Finally, it provides a summary of the total time spent in computation across all training runs. See Figure 2 for a screenshot of the utility.

Additionally, the tool can provide per-training run summaries, showing where each epoch was completed and the amount of execution time taken.

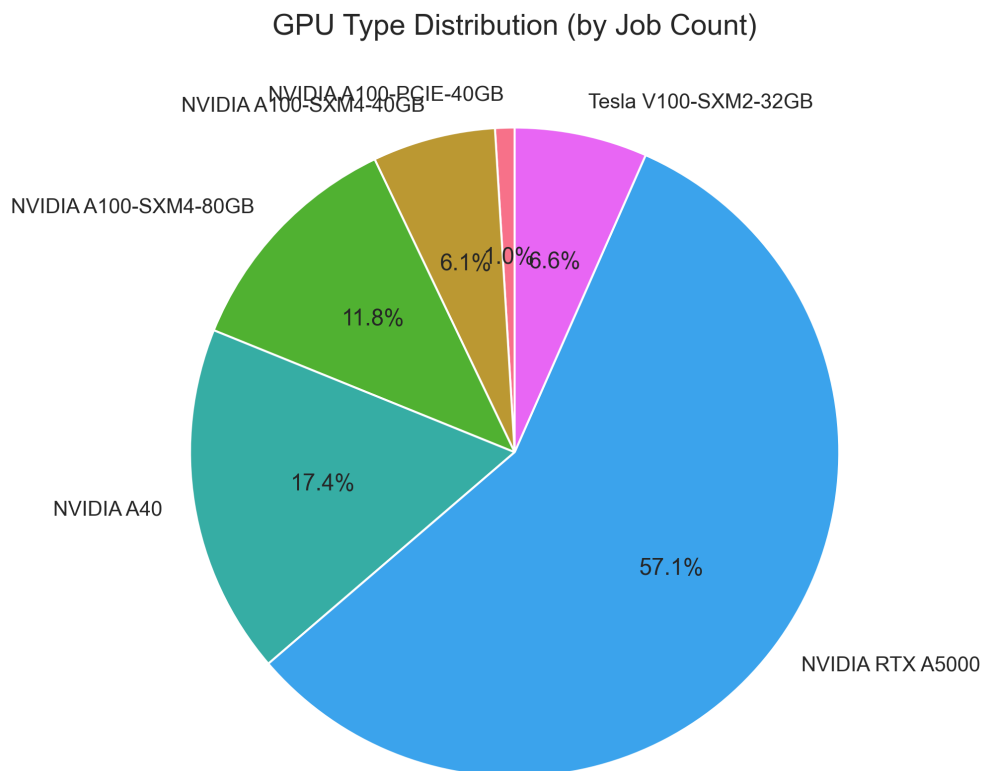


Figure 3: Distribution of epochs by GPU model.

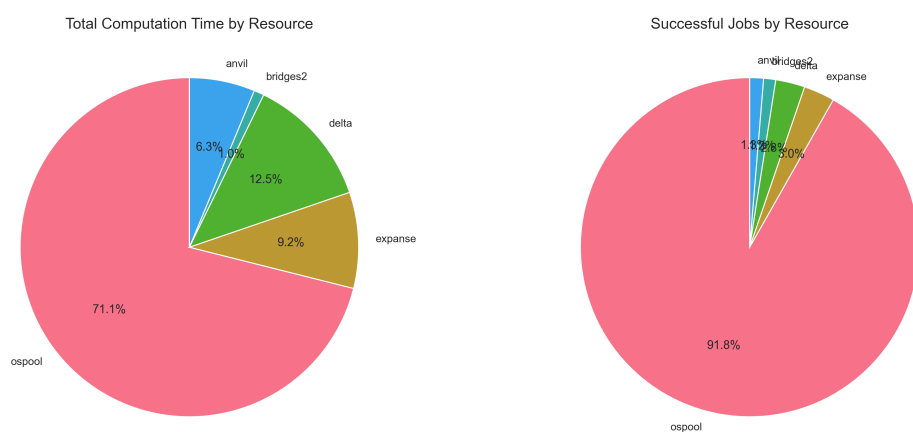


Figure 4: Distribution of training time (left) and epochs trained (right) by targeted resource.

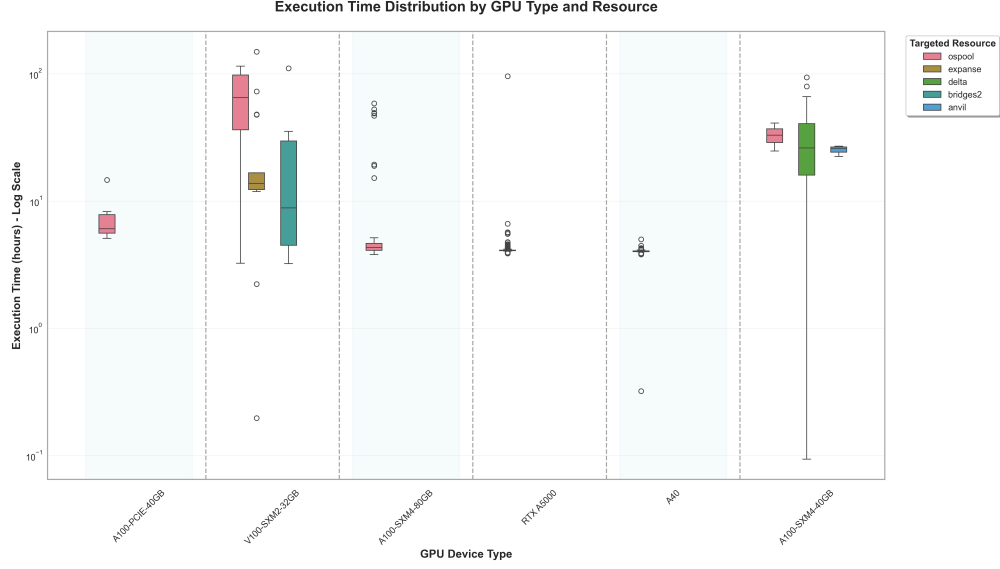


Figure 5: Distribution of training time by GPU type and targeted resource.

4.2 Evaluation

Once training was completed, the model variants were evaluated using validation data. In the validation process, it was discovered that the model configuration utilized an arguments file intended for smaller, more targeted protein language models, while the larger “global” dataset was used as the training data. As a result, the models themselves do not provide scientifically informative predictions, and as a result are only compared to each other for consistency. Training loss and Pearson total scores are shown in Figure 6.

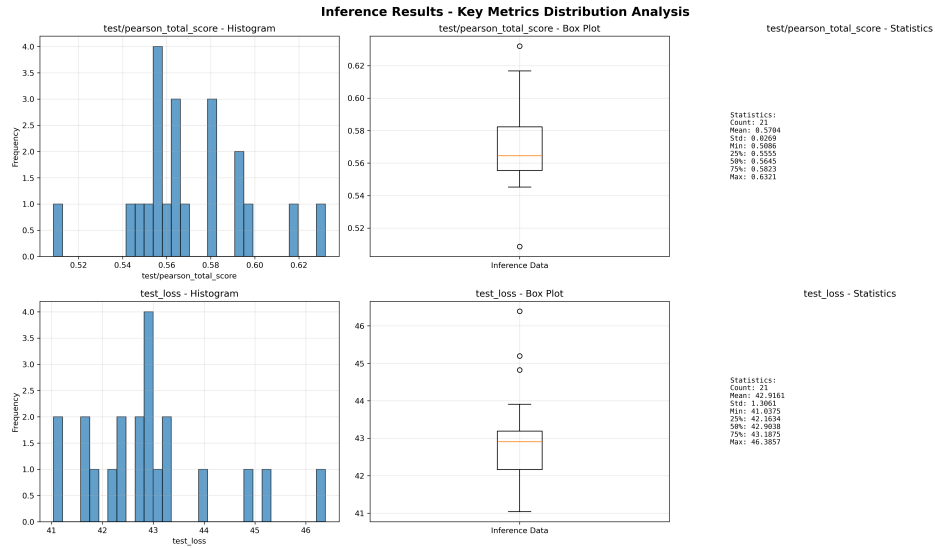


Figure 6: Evaluation results of the 21 model variants. Pictured is comparisons of the Pearson total (top) and test loss (bottom) scores.

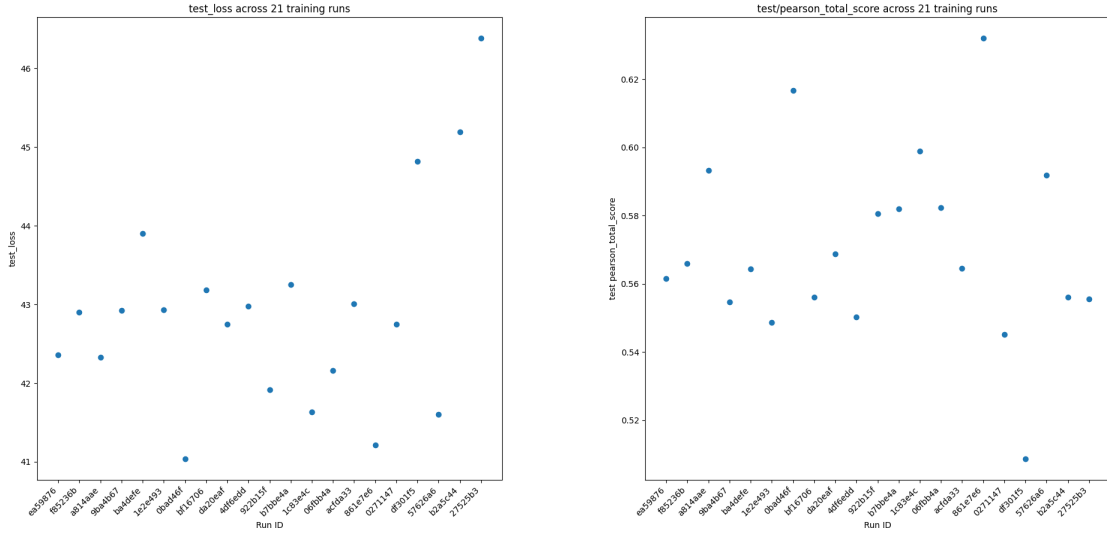


Figure 7: Test loss (left) and Pearson total score (right) for the 21 model variants (by unique identifier of model)

5 Conclusions

5.1 General considerations

For a model of this size (2.5M parameters), the OSPool is an excellent source of GPUs available for training. The limited system and video memory needs are easily available within capacity donated to the OSPool, allowing for efficient training and optimization. It was not atypical for up to a dozen epochs to be training simultaneously, allowing for substantial overall throughput in the training process.

The NAIRR sites also contributed significant computing resources, but they were not as heavily utilized as the OSPool due to the challenges in creating uniform and consistent pressure for jobs to consume existing annexes. The shape of the overall training process (where epochs moved across sites, often with long gaps between re-visiting a NAIRR resource, an annex retired before being re-utilized by additional jobs. This created a substantial bottleneck, as re-establishing the annex required manual intervention (due to the need for two-factor authentication).

The overall behavior of the training process was difficult to characterize, and similar hardware across different sites resulted in significantly different training times. Attempts to address performance variation through CPU/GPU ratio adjustments and data locality improvements yielded minimal gains, suggesting that performance heterogeneity stems from factors beyond job-level resource configuration—potentially network characteristics or underlying hardware variations not captured in GPU model specifications. Troubleshooting attempts included manually logging into the NAIRR sites and submitting the work directly to the SLURM clusters there (to eliminate potential HTCondor/Annex impact), moving data from the job directory to dedicated solid state space before training, and modifying batch sizes.

An innate challenge to adaptation of existing training workflows into a new environment is the unfamiliarity and “black box” nature of the software, which substantially complicates the ability to understand and investigate significant behavioral differences in heterogeneous environments.

5.2 DAG generation and pattern repetition

A pattern that is often repeated in machine learning training is the need to run multiple training jobs with differing configurations. Ensembles of models, including experiments such as this project, and hyperparameter tuning are common uses cases that fall into this category of high numbers of “shish-kebab” shaped

workflows. Between training runs, variables or parameters may be altered in order to optimize the model’s performance, test model architecture behavior, or to generate a group of similar models for comparison.

While creating the DAG generation script, we considered shortcomings and areas for improvement in the HTCondor software, especially in its Directed Acyclic Graph manager (DAGMan). While programmatically creating a DAG in this fashion is not a challenging scripting task, it is largely left to individual researchers to re-implement similar patterns in order to capture desired behavior.

DAGMan provides handles that are useful for this kind of work, such as being able to assign VARS at the NODE level, which allows for easily changing parameters between training runs. However, this must be done at the level of individual nodes, resulting in a verbose overall DAG definition that cannot easily be updated if needed. DAGMan also provides a way to define a subdag, which allows for repeated shapes within the overall workflow, but again lacks the flexibility in changing VARS within repetitions of the shape.

Being able to define subdag- or region-define variables would simplify the overall structure of the DAG file, enabling more concise and maintainable DAG definitions. Finally, hyperparameter sweeps, a very common operation in ML training, involves systematic explorations of a phase space. DAGMan lacks the ability to easily program ”sweeps” of values, such as letting a VAR run from 0.01 to 0.1 in steps of 0.01. Should that feature be added, concise DAG definitions for complex ML training workflows could easily be created and utilized, instead of multiple ala carte solutions being created by research groups.

A concise description of a DAG with SUBDAG repetition and sweeps could look something like:

```
train.subdag
PARENT {run}-train_epoch0 CHILD {run}-train_epoch1
PARENT {run}-train_epoch0 CHILD {run}-eval_epoch1
PARENT {run}-train_epoch1 CHILD {run}-train_epoch2
PARENT {run}-train_epoch1 CHILD {run}-eval_epoch2
VARS {run}-train_epoch0 epoch=1 a={a} b={b}
VARS {run}-train_epoch1 epoch=2 a={a} b={b}
VARS {run}-train_epoch2 epoch=3 a={a} b={b}

workflow.dag
GEN SUBDAG train_flow train.subdag a, b from
    0.01 to 0.1 by 0.01
    0.5 to 5 by 0.5
PARENT init CHILD train_flow
```

This defined a “shish-kebab” training subdag, with parent-child relationships defined between epochs (with sidecar evaluation nodes). a and b represent parameters that change between training runs but are consistent within each run. A GEN command will generate the overall workflow, letting the values of a and b run over their respective ranges. Finally, an initializing node (init, included but not defined) defines any initialization that needs to happen for the workflow as a whole and serves as a PARENT node to the training runs.

5.3 Automation of annex creation

A significant bottleneck in the training process was the challenge in automating the annex creation process. While the DAG seamlessly coordinated scheduling and running epochs within the OSPool, Annex creation as it is currently implemented requires ssh connections to the external resource in order to install, configure, and connect the EP resource to the AP. Because jobs requiring the annex weren’t consistently next in the training runs, the annex would regularly expire before needing to be used for further training. The next time the site was required, the annex needed to be re-established, requiring manual intervention to ssh into the external resource and re-establish the annex.

Although many of these sites are trusted and well-implemented within the OSPool and adjacent computing ecosystems, the Annex was required in order to tie usage to the NAIRR-provided accounts.

5.4 Annex readiness for distributed machine learning training

While HTCCondor Annex provides a unified interface to HPC systems that provide NAIRR allocations, the particular challenge of intentionally shifting training between sites is somewhat at odds with the intended Annex goal of “bursting” into a single HPC site to provide additional short-term computing resources. Instead, there was non-constant pressure of jobs targeting each site, resulting in constant annex retirement and the need for re-identification (often with two-factor authentication) at each site to re-establish the annex and continue training. This prevented full automation of the training process, as manual intervention was often required. Constant pressure (for example, from targeting only one NAIRR resource) could certainly solve this problem, with HTCCondor providing the organization and management aspects of the training run.

5.5 Future work

During the course of this experiment, it was discovered that the model configuration being utilized was inconsistent with the training dataset, resulting in a model that was not scientifically useful and not directly comparable to the existing published versions. A second round of training is underway, utilizing a larger model architecture more suited to the “global” scope of the training data and consistent with the published models. The resources, utilities, and strategies implemented in the initial experiment are being heavily re-used for this second experiment. As of 2025-10-13, the second round of training is underway, with 214 completed epochs of training across 20 different runs.

6 Acknowledgements

This research is supported by the National Artificial Intelligence Research Resource (NAIRR) Pilot. The Anvil supercomputer is supported by the National Science Foundation (award NSF-OAC 2005632) at Purdue University. The Bridges-2 system is supported by NSF award number OAC-1928147 at the Pittsburgh Supercomputing Center (PSC). The Delta advanced computing and data resource is supported by the National Science Foundation (award NSF-OAC 2005572).

References

- [1] Sam Gelman et al. “Biophysics-Based Protein Language Models for Protein Engineering”. In: *Nature Methods* 22.9 (Sept. 2025), pp. 1868–1879. ISSN: 1548-7105. DOI: 10.1038/s41592-025-02776-2. URL: <https://www.nature.com/articles/s41592-025-02776-2> (visited on 10/13/2025).
- [2] Sam Gelman et al. *METL Rosetta datasets*. Zenodo, Feb. 2025. DOI: 10.5281/zenodo.14916528. URL: <https://doi.org/10.5281/zenodo.14916528>.
- [3] Gregory M. Kurtzer et al. *hpcng/singularity: Singularity 3.7.3*. Version v3.7.3. Apr. 2021. DOI: 10.5281/zenodo.4667718. URL: <https://doi.org/10.5281/zenodo.4667718>.